Arrays

- An *array is a collection of variables of the same type that are referred to by a common name.*
- In C#, arrays can have one or more dimensions, although the one-dimensional array is the most common.
- Arrays are used for a variety of purposes because they offer a convenient means of grouping together related variables.
- For example, you might use an array to hold a record of the daily high temperature for a month, a list of stock prices, or your collection of programming books.

- The principal advantage of an array is that it organizes data in such a way that it can be easily manipulated.
- For example, for an array containing the dividends for a selected group of stocks, it is easy to compute the average income by cycling through the array.
- Also, arrays organize data in such a way that it can be easily sorted.

- Although arrays in C# can be used just like arrays in many other programming languages, they have one special attribute:
  - They are implemented as objects.
- By implementing arrays as objects, several important advantages are gained, not the least of which is that unused arrays can be garbage-collected

One-Dimensional Arrays

- A *one-dimensional array is a list of related variables.*
- *Such lists are common in programming.*
- For example, you might use a one-dimensional array to store the account numbers of the active users on a network.
- Another array might store the current batting averages for a baseball team.

- Because arrays in C# are implemented as objects, two steps are needed to obtain an array for use in your program.
- First, declare a variable that can refer to an array.
- Second, create an instance of the array by use of **new.**
- General form:

  type[ ] array-name = new type[size];

- Here, *type declares the element type of the array.*
- *The element type determines the data type* of each element that comprises the array.
- Square brackets that follow *type* indicate that a one-dimensional array is being declared.
- The number of elements that the array will hold is determined by *size.*

Example

- The following creates an int array of ten elements and links it to an array reference variable named **sample.**

```
int[] sample = new int[10];
```

- The sample variable holds a reference to the memory allocated by new.
- This memory is large enough to hold ten elements of type **int.**
- As is the case when creating an instance of a class, it is possible to break the preceding declaration in two.

- For example:

```
int[] sample;
sample = new int[10];
```

- In this case, when sample is first created, it refers to no physical object.
- It is only after the second statement executes that sample refers to an array.
- An individual element within an array is accessed by use of an index.
- An *index describes* the position of an element within an array.

- In C#, all arrays have 0 as the index of their first element.
- Because sample has 10 elements, it has index values of 0 through 9.
- To index an array, specify the number of the element you want, surrounded by square brackets.
- Thus, the first element in sample is sample[0], and the last element is sample[9].
- For example, the following program loads sample with the numbers 0 through 9:

// Demonstrate a one-dimensional array.

```
using System;
class ArrayDemo {
static void Main() {
int[] sample = new int[10];
int i;
for(i = 0; i < 10; i = i+1)
sample[i] = i;
for(i = 0; i < 10; i = i+1)
Console.WriteLine("sample[" + i + "]: " + sample[i]);
}
}
```

The output from the program is shown here:

```
sample[0]: 0
sample[1]: 1
sample[2]: 2
sample[3]: 3
sample[4]: 4
sample[5]: 5
sample[6]: 6
sample[7]: 7
sample[8]: 8
sample[9]: 9
```

Initializing an Array

- Arrays can be initialized when they are created.
- The general form for initializing a onedimensional array is shown here:

  type[ ] array-name = { val1, val2, val3, ..., valN };
- Here, the initial values are specified by *val1 through valN.*
- *They are assigned in sequence,* left to right, in index order.
- C# automatically allocates an array large enough to hold the initializers that you specify.
- There is no need to use the **new operator explicitly.**

Compute the average of a set of values.

```
using System;
class Average {
static void Main() {
int[] nums = { 99, 10, 100, 18, 78, 23,63, 9, 87, 49 };
int avg = 0;
for(int i=0; i < 10; i++)
avg = avg + nums[i];
avg = avg / 10;
Console.WriteLine("Average: " + avg);
}
}
```

Boundaries Are Enforced

- Array boundaries are strictly enforced in C#
- It is a runtime error to overrun or underrun the ends of an array.

- The following program that overruns an array:

```
using System;
class ArrayErr {
static void Main() {
int[] sample = new int[10];
int i;
// Generate an array overrun.
for(i = 0; i < 100; i = i+1)
sample[i] = i;
}
}
```

- As soon as **i reaches 10, an IndexOutOfRangeException is generated and the program is** terminated.

Multidimensional Array

- A multidimensional array is an array that has two or more dimensions, and an individual element is accessed through the combination of two or more indices.

Two-Dimensional Arrays

- The simplest form of the multidimensional array is the two-dimensional array.
- In a two dimensional array, the location of any specific element is specified by two indices.
- A two-dimensional array is a table of information, one index indicates the row, the other indicates the column.

- To declare a two-dimensional integer array table of size 10, 20

  ```
  int[,] table = new int[10, 20];
  ```

- Notice that the two dimensions are separated from each other by a comma.
- In the first part of the declaration, the syntax [,] indicates that a two-dimensional array reference variable is being created.
- When memory is actually allocated for the array using new, this syntax is used:

  ```
  int[10, 20]
  ```

- This creates a 10×20 array, and again, the comma separates the dimensions.

- To access an element in a two-dimensional array, you must specify both indices, separating the two with a comma.
- For example, to assign the value 10 to location 3, 5 of array table, you would use table[3, 5] = 10;

Example

- It loads a two-dimensional array with the numbers 1 through 12 and then displays the contents of the array.

```
using System;
class TwoD {
static void Main() {
int t, i;
int[,] table = new int[3, 4];
for(t=0; t < 3; ++t) {
for(i=0; i < 4; ++i) {
table[t,i] = (t*4)+i+1;
Console.Write(table[t,i] + " ");
}
Console.WriteLine();
}
}
}
```

- In this example, table[0, 0] will have the value 1, table[0, 1] the value 2, table[0, 2] the value 3, and so on.
- The value of table[2, 3] will be 12.

- *If you have previously programmed in C, C++, or Java, be careful when declaring or accessing multidimensional arrays in C#.*
- *In these other languages, array dimensions and indices are specified within their own set of brackets.*
- *C# separates dimensions using commas.*

Arrays of Three or More Dimensions

- C# allows arrays with more than two dimensions.
- Here is the general form of a multidimensional array declaration:

  type[, ...,] name = new type[size1, size2, ..., sizeN];

- For example, the following declaration creates a 4×10×3 three-dimensional integer array:

```
int[,,] multidim = new int[4, 10, 3];
```

- To assign element 2, 4, 1 of multidim the value 100, use this statement:

```
multidim[2, 4, 1] = 100;
```

- Here is a program that uses a three-dimensional array that holds a 3×3×3 matrix of values.
- It then sums the value on one of the diagonals through the cube.
- // Sum the values on a diagonal of a 3x3x3 matrix.

```
using System;
class ThreeDMatrix {
static void Main() {
int[,,] m = new int[3, 3, 3];
int sum = 0;
int n = 1;
for(int x=0; x < 3; x++)
for(int y=0; y < 3; y++)
for(int z=0; z < 3; z++)
m[x, y, z] = n++;
sum = m[0, 0, 0] + m[1, 1, 1] + m[2, 2, 2];
Console.WriteLine("Sum of first diagonal: " + sum);
}
}
```

- The output is shown here: Sum of first diagonal: 42

Initializing Multidimensional Arrays

- A multidimensional array can be initialized by enclosing each dimension's initializer list within its own set of curly braces.
- For example, the general form of array initialization for a two-dimensional array is :

  type[,] array_name = {
  { *val, val, val, ..., val },*
  { *val, val, val, ..., val },*
  ...
  { *val, val, val, ..., val }*
  };

- Here, *val indicates an initialization value.*
- *Each inner block designates a row.*
- *Within each* row, the first value will be stored in the first position, the second value in the second position, and so on.
- Notice that commas separate the initializer blocks and that a semicolon follows the closing **}.**

Example, program to initialize an array called sqrs with the numbers 1 through 10 and their squares.

```
using System;
class Squares {
static void Main() {
int[,] sqrs = {
{ 1, 1 },
{ 2, 4 },
{ 3, 9 },
{ 4, 16 },
{ 5, 25 },
{ 6, 36 },
{ 7, 49 },
{ 8, 64 },
{ 9, 81 },
{ 10, 100 }
};
int i, j;
for(i=0; i < 10; i++) {
for(j=0; j < 2; j++)
Console.Write(sqrs[i,j] + " ");
Console.WriteLine();
}
}
}
```

- Here is the output from the program:
- 1 1
- 2 4
- 3 9
- 4 16
- 5 25
- 6 36
- 7 49
- 8 64
- 9 81
- 10 100

Jagged Arrays

- In the preceding examples, when you created a two-dimensional array, you were creating what C# calls a rectangular array.
- Thinking of two-dimensional arrays as tables, a rectangular array is a two-dimensional array in which the length of each row is the same for the entire array.
- However, C# also allows you to create a special type of two-dimensional array called a jagged array.

- A jagged array is an array of arrays in which the length of each array can differ.
- Thus, a jagged array can be used to create a table in which the lengths of the rows are not the same.

- Jagged arrays are declared by using sets of square brackets to indicate each dimension.
- For example, to declare a two-dimensional jagged array, you will use this general form:

  type[ ] [ ] array-name = new type[size][ ];
- Here, *size indicates the number of rows in the array.*
- *The rows, themselves, have not been* allocated.
- Instead, the rows are allocated individually.
- This allows for the length of each row to vary.

- For example, the following code allocates memory for the first dimension of **jagged** when it is declared.
- It then allocates the second dimensions manually.

```
int[][] jagged = new int[3][];
jagged[0] = new int[4];
jagged[1] = new int[3];
jagged[2] = new int[5];
```

- Once a jagged array has been created, an element is accessed by specifying each index within its own set of brackets.
- For example, to assign the value 10 to element 2, 1 of **jagged**:

```
jagged[2][1] = 10;
```

- Note that this differs from the syntax that is used to access an element of a rectangular array.

Jagged two-dimensional array:

```
using System;
class Jagged {
static void Main() {
int[][] jagged = new int[3][];
jagged[0] = new int[4];
jagged[1] = new int[3];
jagged[2] = new int[5];
int i;
// Store values in first array.
for(i=0; i < 4; i++)
jagged[0][i] = i;
// Store values in second array.
for(i=0; i < 3; i++)
jagged[1][i] = i;
```

```
// Store values in third array.
for(i=0; i < 5; i++)
jagged[2][i] = i;
// Display values in first array.
for(i=0; i < 4; i++)
Console.Write(jagged[0][i] + " ");
Console.WriteLine();
// Display values in second array.
for(i=0; i < 3; i++)
Console.Write(jagged[1][i] + " ");
Console.WriteLine();
// Display values in third array.
for(i=0; i < 5; i++)
Console.Write(jagged[2][i] + " ");
Console.WriteLine();
}
}
```

- The output is shown here:
- 0 1 2 3
- 0 1 2
- 0 1 2 3 4

- Jagged arrays are not used by all applications, but they can be effective in some situations.
- For example, if you need a very large two-dimensional array that is sparsely populated (that is, one in which not all of the elements will be used), then a jagged array might be a perfect solution.
- Because jagged arrays are arrays of arrays, there is no restriction that requires that the arrays be one-dimensional.
- For example, the following creates an array of two-dimensional arrays:

```
int[][,] jagged = new int[3][,];
```

- The next statement assigns jagged[0] a reference to a 4×2 array:

```
jagged[0] = new int[4, 2];
```

- The following statement assigns a value to jagged[0][1,0]:

```
jagged[0][1,0] = i;
```

```
class MyjaggedArr
{
public static void Main()
{
int[][] myjag=new int[3][];
for(int i=0;i<myjag.Length;i++)
{
myjag[i]=new int[i+3];
}
for(int i=0;i<3;i++)
{
Console.WriteLine("Enter the Elements of row{0}",i);
for(int j=0;j<myjag[i].Length;j++)
{
myjag[i][j]=int.Parse(Console.ReadLine());
}
}
int sum=0;
for(int i=0;i<3;i++)
{
for(int j=0;j<myjag[i].Length;j++)
{
sum+=myjag[i][j];
}
}
Console.WriteLine("sum="+sum);
}
}
```

System.Array

- Provides methods for creating, manipulating, searching, and sorting arrays, thereby serving as the base class for all arrays in the common language runtime.
- **Inheritance Hierarchy**
- System.Object
  System.Array

Properties

- Next Slide

Name	Description
IsFixedSize	Gets a value indicating whether the Array has a fixed size.
IsReadOnly	Gets a value indicating whether the Array is read-only.
IsSynchronized	Gets a value indicating whether access to the Array is synchronized (thread safe).
Length	Gets a 32-bit integer that represents the total number of elements in all the dimensions of the Array.
LongLength	Gets a 64-bit integer that represents the total number of elements in all the dimensions of the Array.
Rank	Gets the rank (number of dimensions) of the Array. For example, a one-dimensional array returns 1, a two-dimensional array returns 2, and so on.
SyncRoot	Gets an object that can be used to synchronize access to the Array.

Methods

- Next Slide

Name	Description
AsReadOnly<T>	Returns a read-only wrapper for the specified array.
BinarySearch(Array, Object)	Searches an entire one-dimensional sorted array for a specific element, using the IComparable interface implemented by each element of the array and by the specified object.
BinarySearch(Array, Object, IComparer)	Searches an entire one-dimensional sorted array for a value using the specified IComparer interface.
BinarySearch(Array, Int32, Int32, Object)	Searches a range of elements in a one-dimensional sorted array for a value, using the IComparable interface implemented by each element of the array and by the specified value.
BinarySearch(Array, Int32, Int32, Object, IComparer)	Searches a range of elements in a one-dimensional sorted array for a value, using the specified IComparer interface.
BinarySearch<T>(T[], T)	Searches an entire one-dimensional sorted array for a specific element, using the IComparable<T> generic interface

BinarySearch<T>(T[], T, IComparer<T>)	**Searches an entire one-dimensional sorted array for a value using the specified IComparer<T> generic interface.**
BinarySearch<T>(T[], Int32, Int32, T)	Searches a range of elements in a one-dimensional sorted array for a value, using the IComparable<T> generic interface implemented by each element of the Array and by the specified value.
BinarySearch<T>(T[], Int32, Int32, T, IComparer<T>)	Searches a range of elements in a one-dimensional sorted array for a value, using the specified IComparer<T> generic interface.
Clear	Sets a range of elements in an array to the default value of each element type.
Clone	Creates a shallow copy of the Array.
ConstrainedCopy	Copies a range of elements from an Array starting at the specified source index and pastes them to another Array starting at the specified destination index. Guarantees that all changes are undone if the copy does not succeed completely.
ConvertAll<TInput, TOutput>	Converts an array of one type to an array of another type.
	Copies a range of elements from an Array starting at the

Copy(Array, Array, Int64)	**Copies a range of elements from an Array starting at the first element and pastes them into another Array starting at the first element. The length is specified as a 64-bit integer.**
Copy(Array, Int32, Array, Int32, Int32)	Copies a range of elements from an Array starting at the specified source index and pastes them to another Array starting at the specified destination index. The length and the indexes are specified as 32-bit integers.
Copy(Array, Int64, Array, Int64, Int64)	Copies a range of elements from an Array starting at the specified source index and pastes them to another Array starting at the specified destination index. The length and the indexes are specified as 64-bit integers.
CopyTo(Array, Int32)	Copies all the elements of the current one-dimensional array to the specified one-dimensional array starting at the specified destination array index. The index is specified as a 32-bit integer.
CopyTo(Array, Int64)	Copies all the elements of the current one-dimensional array to the specified one-dimensional array starting at the specified

CreateInstance(Type, Int32)	Creates a one-dimensional Array of the specified Type and length, with zero-based indexing.
CreateInstance(Type, Int32[])	Creates a multidimensional Array of the specified Type and dimension lengths, with zero-based indexing. The dimension lengths are specified in an array of 32-bit integers.
CreateInstance(Type, Int64[])	Creates a multidimensional Array of the specified Type and dimension lengths, with zero-based indexing. The dimension lengths are specified in an array of 64-bit integers.
CreateInstance(Type, Int32, Int32)	Creates a two-dimensional Array of the specified Type and dimension lengths, with zero-based indexing.
CreateInstance(Type, Int32[], Int32[])	Creates a multidimensional Array of the specified Type and dimension lengths, with the specified lower bounds.
CreateInstance(Type, Int32, Int32, Int32)	Creates a three-dimensional Array of the specified Type and dimension lengths, with zero-based indexing.

Empty<T>	Returns an empty array.
Equals(Object)	Determines whether the specified object is equal to the current object. (Inherited from Object.)
Exists<T>	Determines whether the specified array contains elements that match the conditions defined by the specified predicate.
Finalize	Allows an object to try to free resources and perform other cleanup operations before it is reclaimed by garbage collection. (Inherited from Object.)
Find<T>	Searches for an element that matches the conditions defined by the specified predicate, and returns the first occurrence within the entire Array.
FindAll<T>	Retrieves all the elements that match the conditions defined by the specified predicate.

FindIndex<T>(T[], Predicate<T>)	**Searches for an element that matches the conditions defined by the specified predicate, and returns the zero-based index of the first occurrenc within the entire Array.**
FindIndex<T>(T[], Int32, Predicate<T>)	Searches for an element that matches the condition defined by the specified predicate, and returns the zero-based index of the first occurrence within the range of elements in the Array that extends from th specified index to the last element.
FindIndex<T>(T[], Int32, Int32, Predicate<T>)	Searches for an element that matches the condition defined by the specified predicate, and returns the zero-based index of the first occurrence within the range of elements in the Array that starts at the specified index and contains the specified number elements.
FindLast<T>	Searches for an element that matches the condition defined by the specified predicate, and returns the last occurrence within the entire Array.
FindLastIndex<T>(T[], Predicate<T>)	Searches for an element that matches the condition defined by the specified predicate, and returns the zero-based index of the last occurrence within the entire Array.

FindLastIndex<T>(T[], Int32, Int32, Predicate<T>)	**Searches for an element that matches the conditions defined by the specified predicate, and returns the zero-based index of the last occurrence within the range of elements in the Array that contains the specified number of elements and ends at the specified index.**
ForEach<T>	Performs the specified action on each element of the specified array.
GetEnumerator	Returns an IEnumerator for the Array.
GetHashCode	Serves as the default hash function. (Inherited from Object.)
GetLength	Gets a 32-bit integer that represents the number of elements in the specified dimension of the Array.
GetLongLength	Gets a 64-bit integer that represents the number of elements in the specified dimension of the Array.

GetLowerBound	**Gets the index of the first element of the specified dimension in the array.**
GetType	Gets the Type of the current instance. (Inherited from Object.)
GetUpperBound	Gets the index of the last element of the specified dimension in the array.
GetValue(Int32)	Gets the value at the specified position in the one-dimensional Array. The index is specified as a 32-bit integer.
GetValue(Int32[])	Gets the value at the specified position in the multidimensional Array. The indexes are specified as an array of 32-bit integers.
GetValue(Int64)	Gets the value at the specified position in the one-dimensional Array. The index is specified as a 64-bit integer.

GetValue(Int64[])	**Gets the value at the specified position in the multidimensional Array. The indexes are specified as an array of 64-bit integers.**
GetValue(Int32, Int32)	Gets the value at the specified position in the two-dimensional Array. The indexes are specified as 32-bit integers.
GetValue(Int64, Int64)	Gets the value at the specified position in the two-dimensional Array. The indexes are specified as 64-bit integers.
GetValue(Int32, Int32, Int32)	Gets the value at the specified position in the three-dimensional Array. The indexes are specified as 32-bit integers.
GetValue(Int64, Int64, Int64)	Gets the value at the specified position in the three-dimensional Array. The indexes are specified as 64-bit integers.

IndexOf(Array, Object)	Searches for the specified object and returns the index of its first occurrence in a one-dimensional array.
IndexOf(Array, Object, Int32)	Searches for the specified object in a range of elements of a one-dimensional array, and returns the index of its first occurrence. The range extends from a specified index to the end of the array.
IndexOf(Array, Object, Int32, Int32)	Searches for the specified object in a range of elements of a one-dimensional array, and returns the index of ifs first occurrence. The range extends from a specified index for a specified number of elements.
IndexOf<T>(T[], T)	Searches for the specified object and returns the index of its first occurrence in a one-dimensional array.
IndexOf<T>(T[], T, Int32)	Searches for the specified object in a range of elements of a one dimensional array, and returns the index of its first occurrence. The range extends from a specified index to the end of the array.
	Searches for the specified object in a range of

Initialize	Initializes every element of the value-type Array by calling the default constructor of the value type.
LastIndexOf(Array, Object)	Searches for the specified object and returns the index of the last occurrence within the entire one-dimensional Array.
LastIndexOf(Array, Object, Int32)	Searches for the specified object and returns the index of the last occurrence within the range of elements in the one-dimensional Array that extends from the first element to the specified index.
LastIndexOf(Array, Object, Int32, Int32)	Searches for the specified object and returns the index of the last occurrence within the range of elements in the one-dimensional Array that contains the specified number of elements and ends at the specified index.
LastIndexOf<T>(T[], T)	Searches for the specified object and returns the index of the last occurrence within the entire Array.

LastIndexOf<T>(T[], T, Int32)	**Searches for the specified object and returns the index of the last occurrence within the range of elements in the Array that extends from the first element to the specified index.**
LastIndexOf<T>(T[], T, Int32, Int32)	Searches for the specified object and returns the index of the last occurrence within the range of elements in the Array that contains the specified number of elements and ends at the specified index.
MemberwiseClone	Creates a shallow copy of the current Object. (Inherited from Object.)
Resize<T>	Changes the number of elements of a one-dimensional array to the specified new size.
Reverse(Array)	Reverses the sequence of the elements in the entire one-dimensional Array.

Reverse(Array, Int32, Int32)	**Reverses the sequence of the elements in a range of elements in the one-dimensional Array.**
SetValue(Object, Int32)	Sets a value to the element at the specified position in the one-dimensional Array. The index is specified as a 32-bit integer.
SetValue(Object, Int32[])	Sets a value to the element at the specified position in the multidimensional Array. The indexes are specified as an array of 32-bit integers.
SetValue(Object, Int64)	Sets a value to the element at the specified position in the one-dimensional Array. The index is specified as a 64-bit integer.
SetValue(Object, Int64[])	Sets a value to the element at the specified position in the multidimensional Array. The indexes are specified as an array of 64-bit integers

SetValue(Object, Int32, Int32)	**Sets a value to the element at the specified position in the two-dimensional Array. The indexes are specified as 32-bit integers.**
SetValue(Object, Int64, Int64)	Sets a value to the element at the specified position in the two-dimensional Array. The indexes are specified as 64-bit integers.
SetValue(Object, Int32, Int32, Int32)	Sets a value to the element at the specified position in the three-dimensional Array. The indexes are specified as 32-bit integers.
SetValue(Object, Int64, Int64, Int64)	Sets a value to the element at the specified position in the three-dimensional Array. The indexes are specified as 64-bit integers.
Sort(Array)	Sorts the elements in an entire one-dimensional Array using the IComparable implementation of each element of the Array.

Extension Methods

AsParallel	**Enables parallelization of a query. (Defined by ParallelEnumerable.)**
AsQueryable	Converts an IEnumerable to an IQueryable. (Defined by Queryable.)
Cast<TResult>	Casts the elements of an IEnumerable to the specified type. (Defined by Enumerable.)
OfType<TResult>	Filters the elements of an IEnumerable based on a specified type. (Defined by Enumerable.)

Explicit Interface Implementations

ICollection.Count	**Gets the number of elements contained in the Array.**
ICollection.IsSynchronized	Gets a value that indicates whether access to the Array is synchronized (thread safe).
ICollection.SyncRoot	Gets an object that can be used to synchronize access to the Array.
IList.Add	Calling this method always throws a NotSupportedException exception.
IList.Clear	Removes all items from the IList.

IList.Contains	**Determines whether an element is in the IList.**
IList.IndexOf	Determines the index of a specific item in the IList.
IList.Insert	Inserts an item to the IList at the specified index.
IList.IsFixedSize	Gets a value that indicates whether the Array has a fixed size.
IList.IsReadOnly	Gets a value that indicates whether the Array is read-only.

IList.Item	**Gets or sets the element at the specified index.**
IList.Remove	Removes the first occurrence of a specific object from the IList.
IList.RemoveAt	Removes the IList item at the specified index.
IStructuralComparable.CompareTo	Determines whether the current collection object precedes, occurs in the same position as, or follows another object in the sort order.
IStructuralEquatable.Equals	Determines whether an object is equal to the current instance.
IStructuralEquatable.GetHashCode	Returns a hash code for the current instance.

Collections

- For many applications, we may want to create and manage groups of related objects.
- There are two ways to group objects:
  - by creating arrays of objects, and
  - by creating collections of objects.
- Arrays are most useful for creating and working with a fixed number of strongly-typed objects.

- Collections provide a more flexible way to work with groups of objects.
- Unlike arrays, the group of objects we work with can grow and shrink dynamically as the needs of the application change.
- For some collections, we can assign a key to any object that we put into the collection so that we can quickly retrieve the object by using the key.

- A collection is a class, so we must declare a new collection before we can add elements to that collection.
- If the collection contains elements of only one data type, we can use one of the classes in the System.Collections.Generic namespace.
- A generic collection enforces type safety so that no other data type can be added to it.
- When we retrieve an element from a generic collection, we do not have to determine its data type or convert it.

Kinds of Collections

- Many common collections are provided by the .NET Framework.
- Each type of collection is designed for a specific purpose.

  System.Collections.Generic classes

  System.Collections.Concurrent classes

  System.Collections classes

System.Collections.Generic Classes

- We can create a generic collection by using one of the classes in the System.Collections.Generic namespace.
- A generic collection is useful when every item in the collection has the same data type.
- A generic collection enforces strong typing by allowing only the desired data type to be added.

The following table lists some of the frequently used classes of the System.Collections.Generic namespace:

Class	Description
Dictionary<TKey, TValue>	Represents a collection of key/value pairs that are organized based on the key.
List<T>	Represents a list of objects that can be accessed by index. Provides methods to search, sort, and modify lists.
Queue<T>	Represents a first in, first out (FIFO) collection of objects.
SortedList<TKey, TValue>	Represents a collection of key/value pairs that are sorted by key based on the associated IComparer<T> implementation.
Stack<T>	Represents a last in, first out (LIFO) collection of objects.

System.Collections Classes

- The classes in the System.Collections namespace do not store elements as specifically typed objects, but as objects of type Object.
- Whenever possible, we should use the generic collections in the System.Collections.Generic namespace or the System.Collections.Concurrent namespace instead of the legacy types in the **System.Collections** namespace.

The following table lists some of the frequently used classes in the **System.Collections** namespace:

Class	Description
ArrayList	Represents an array of objects whose size is dynamically increased as required.
Hashtable	Represents a collection of key/value pairs that are organized based on the hash code of the key.
Queue	Represents a first in, first out (FIFO) collection of objects.
Stack	Represents a last in, first out (LIFO) collection of objects.